# Checkerboard Chunking approach to Parallel Wave Function Collapse

Seminar paper

:

Ostrava, 2026

# Contents

**List of symbols and abbreviations** **3**

**List of Figures** **4**

**List of Tables** **5**

**1 Introduction** **6**

**2 Sequential Wave Function Collapse** **7**

**3 Existing Solutions** **11**

**4 Proposed Solution** **12**

4.1 Considered Approaches . . . . . 12

4.2 Checkerboard Decomposition . . . . . 13

**5 Implementation** **17**

**6 Experiments** **22**

6.1 Experimental Setup . . . . . 22

6.2 Metrics . . . . . 23

6.3 Results . . . . . 23

**7 Conclusion** **29**

**Literatura** **30**

# List of symbols and abbreviations

| | | |
|---|---|---|
| WFC | – | Wave Function Collapse |

# List of Figures


2.1 Examples of images generated using Wave Function Collapse provided by Maxim Gumin [2] . . . . . 7
2.2 Summer Tileset provided by Maxim Gumin [2] . . . . . 8
2.3 Propagation example provided by Maxim Gumin [2] . . . . . 9

4.1 Checkboard Pattern . . . . . 14

6.1 Circuit Tileset provided by Maxim Gumin [2] . . . . . 23
6.2 Sequential and parallel generation time through testing passes . . . . . 24
6.3 Speedup of parallel solution compared to sequential one, relative to grid size . . . . . 25
6.4 Oversubscription influence on performance . . . . . 26
6.5 Performance of parallel and sequential algorithms on a 150x150 grid . . . . . 27

# List of Tables

6.1 Experimental hardware configuration. . . . . 22

# Chapter 1

# Introduction

**Stylistic corrections, as well as grammar control was performed using AI.**

The goal of this paper is to present a checkerboard chunk approach to parallelizing Wave Function Collapse (WFC in the future).

In the first chapter a sequential WFC algorithm is presented, as it is necessary for understanding of parallel solution. Additionally specifications on provided solutions are presented, adding context to the proposed algorithm and comparison to existing solutions.

In the second chapter, existing parallel approaches are presented and analyzed.

In the third chapter a proposed solution, including pseudocode and math are provided, along with the reasoning.

In the fourth chapter experiments are conducted to analyze performance gain from a proposed parallel algorithm in comparison to a specified sequential solution.

In the last chapter a broad overview of achieved results are presented, as well as limitations and potential improvements.

# Chapter 2

# Sequential Wave Function Collapse

**Wave Function Collapse (WFC)** is a family of constraint-satisfaction algorithms widely used in procedural content generation, particularly in the video game industry, for the synthesis of images, terrain, and levels. The algorithm is rooted in the **Model Synthesis** framework introduced by Paul Merrell in his doctoral dissertation [1], and was later adapted to a 2D tile-based representation by Maxim Gumin, who also proposed a data-driven approach in which local adjacency rules are derived automatically from a small sample image [2].

The name *Wave Function Collapse* draws an analogy between the algorithm's mechanism of applying `local constraints` across the grid and the observation (measurement) process in quantum mechanics, where a superposition of states collapses to a single definite state.

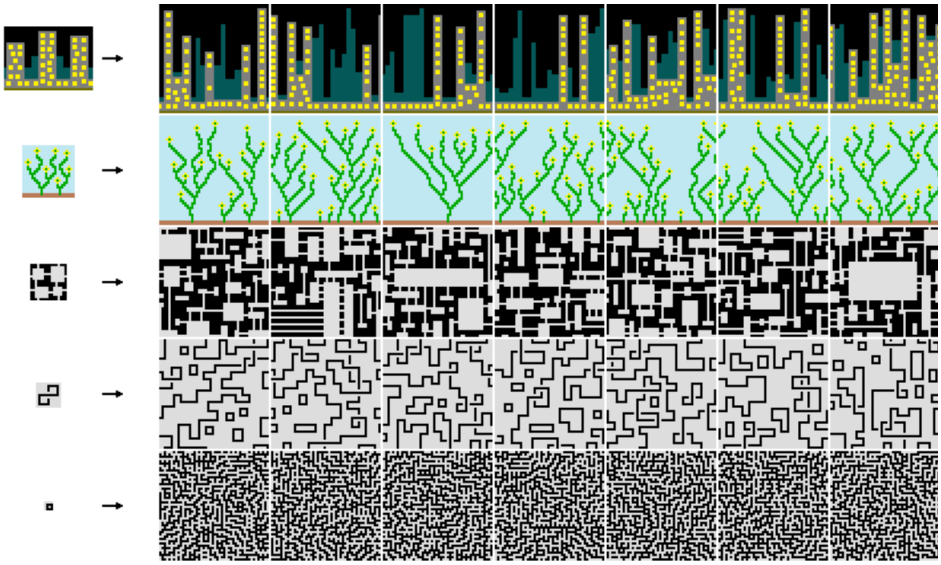

Figure 2.1: Examples of images generated using Wave Function Collapse provided by Maxim Gumin [2]

## Tile Decomposition and Constraint Extraction

WFC operates by decomposing a provided sample image into a finite set of tiles and inferring a set of *adjacency rules* that define which tiles may appear next to one another. It is important to note that the tileset and its rules may also be defined manually by developers, enabling flexible and domain-specific content generation.

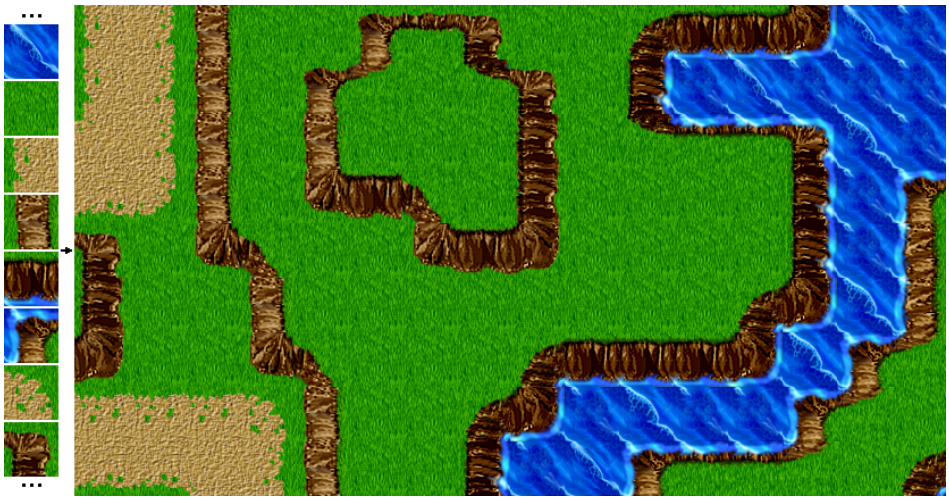


Figure 2.2: Summer Tileset provided by Maxim Gumin [2]

After the tileset is derived, a grid of the desired output dimensions is created. Each cell maintains two fields: a set of *possible tiles* and an *entropy* value. The set of possible tiles represents all states into which the cell may be collapsed while still satisfying the constraints propagated from neighboring cells. Entropy is interpreted in the sense of Shannon information theory as a measure of uncertainty: a cell with a single possible tile has entropy 1 (fully determined), whereas a cell whose entire tileset remains possible has maximum entropy. A cell with zero possible tiles represents a *contradiction*, indicating that no valid assignment exists under the current constraints.

The grid is initialized with every cell at maximum entropy. Generation begins by selecting a cell at random and **collapsing** it - that is, randomly choosing one tile from its set of possible options and discarding all others, reducing its entropy to 1 and marking the cell as *defined*.

Following initialization, the algorithm enters its main generative loop. Generation proceeds by repeatedly applying two steps - *observation* and *propagation* - until every cell in the grid has been collapsed to a single tile, yielding the final output image.

1. **Observation:** The cell with the lowest entropy is selected and collapsed - that is, one tile is chosen at random from its set of remaining possibilities, and all other possibilities are discarded. Ties in entropy are broken randomly.

2. **Propagation:** The consequences of the collapse are propagated across the grid. Starting from the newly collapsed cell, any neighbor whose set of possible tiles is inconsistent with the new state of the cell has its possibilities reduced accordingly. This reduction may in turn affect further neighbors, so propagation continues until no cell's possibility set changes - that is, until the grid reaches a locally consistent state.

These two steps are repeated until all cells are collapsed. If at any point a cell's possibility set becomes empty, a *contradiction* has occurred, and the algorithm must either backtrack or restart.

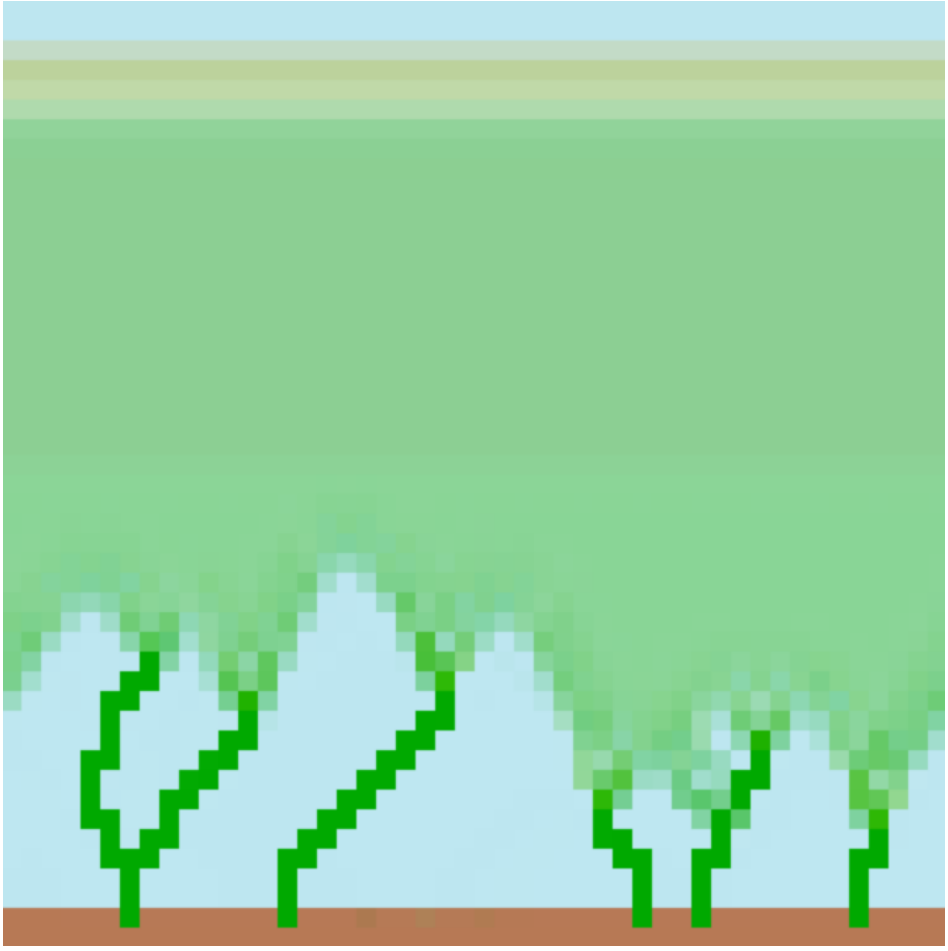

Figure 2.3: Propagation example provided by Maxim Gumin [2]

## Propagation Strategies

Several approaches exist for propagating constraints following an observation step. The simplest strategy — *full-grid traversal*, re-evaluates every cell in the grid after each observation, updating each cell's possibility set based on the current states of its neighbors [3]. While correct, this approach is inefficient: most cells are unaffected by a given collapse, yet all are visited regardless.

The implementation used in this work adopts a more efficient alternative based on *breadth-first propagation*. A queue is seeded with the most recently collapsed cell, and a neighbor is enqueued only if its possibility set was reduced as a result of processing the current cell. Constraint updates therefore spread outward from the observation point as a wavefront, and propagation terminates naturally once no further reductions occur. Although changes can theoretically reach the entire grid, in practice the wavefront dissipates after a limited radius, making breadth-first propagation substantially faster than full-grid traversal in the average case.

The distinction between these two strategies will be revisited in Chapter 4, where the choice of propagation method has direct implications for parallelization.

# Chapter 3

# Existing Solutions

Wave Function Collapse is a relatively recent algorithm, and attempts to introduce parallelism remain scarce in the literature.

## Parallel Wave Function Collapse (Orlowski & Lee)

The most relevant prior work is the *Parallel Wave Function Collapse* project by Jan Orlowski and Amy Lee [3]. Profiling of the sequential implementation revealed that the propagation step accounts for the majority of total runtime, making it the natural target for parallelization. Two principal approaches were explored.

**Full-grid traversal parallelization.** In the first approach, multiple threads were assigned to cells of the grid, each independently comparing its cell's state against its neighbors and updating the possibility set accordingly. This yielded a speedup of approximately 2–4× over the sequential baseline. However, the inherent inefficiency of scanning the entire grid on every propagation step remained a bottleneck, limiting further gains.

**Breadth-first propagation parallelization.** The second approach parallelized the BFS-based propagation described in Chapter 2, using a lock-free queue to allow multiple threads to dequeue and process cells concurrently. This avoided redundant cell visits and achieved higher throughput than full-grid traversal, though contention on the shared queue introduced its own overhead at higher thread counts.

This approach lead to similar performance gain as the solution described in this work, so comparison of those methods if left for future investigation.

# Chapter 4

# Proposed Solution

Parallelizing Wave Function Collapse is not straightforward. The algorithm relies on sequential constraint propagation to maintain consistency across the grid, meaning that a collapse in one cell can affect distant cells through a chain of propagation steps, and there is no heuristic to accurately estimate how far the propagation will go. Naively assigning multiple threads to collapse cells simultaneously introduces race conditions, as concurrent writes to shared cell state during propagation produce. Any viable parallel approach must therefore either eliminate propagation dependencies between threads, or manage them explicitly, while keeping synchronization costs as low as possible.

This chapter analyzed possible sollutions and derives resulting approach, which addresses the problems mentioned above.

## 4.1 Considered Approaches

### Simultaneous Cell Collapse

The most direct approach is to collapse multiple cells simultaneously, assigning one thread per cell and running observation and propagation concurrently. This is, however, fundamentally unsafe: propagation from two simultaneously collapsed cells may attempt to modify the same neighboring cell, causing a data race. Resolving this would require locking individual cells during propagation, which serializes execution. However, it is important to note that locking individual cells could be a limiting factor in performance if algorithm design minimizes race conditions by managing propagation, which will be addressed later in a final solution.

### Rule Propagation Parallelization

An alternative is to leave the outer observation-propagation loop sequential, and instead parallelize the inner loop that evaluates adjacency rule compatibility for a single cell. Early in generation,

a cell may retain fifty or more candidate tiles, depending on the tileset, making the compatibility checks across those candidates a reasonable target for parallelization. However, this opportunity diminishes as generation progresses: as cells collapse, the average candidate count decays toward one, and thread overhead begins to outweigh the parallelism gained. This approach is therefore most useful when the grid is too small for spatial decomposition to be effective, and degrades in utility as entropy decreases.

### Chunk-Based Decomposition

A more promising direction is to partition the output grid into spatial chunks and assign each chunk to an independent thread. If chunks can be made truly independent - that is, if no propagation needs to cross chunk boundaries during generation - then threads require no synchronization and the approach scales cleanly with thread count.

The difficulty lies in ensuring boundary consistency. If chunks are generated in isolation with no knowledge of their neighbors, the tiles at shared edges are likely to be incompatible, producing conflicts which would require complex multi-thread backtracking, or overall image regeneration. Several chunk orderings were considered to solve this:

- **Random ordering** offers no consistency guarantees and produces unpredictable conflict rates.
- **Lowest-entropy-first ordering** prioritizes the most constrained chunks, but introduces a synchronization overhead, as each chunk needs to be generated not only with awareness of existing chunks, but parts generated by other threads, reducing effectiveness of the algorithm.
- **Highest-entropy-first ordering** defers the most constrained chunks to last, where accumulated boundary information reduces the risk of contradiction. However, the final chunks still lack fully determined borders, leaving conflict resolution unresolved. Additionally it does not solve the problem of thread progress synchronization, as it still requires to know what is the current state of neighboring chunks, thus increasing race conditions probability.

None of these orderings guarantees that a chunk's borders are fully determined before it begins generation, while maintaining thread independence, which is the root of the consistency problem.

## 4.2 Checkerboard Decomposition

The approach adopted in this work resolves the boundary consistency problem through a structured two-pass decomposition. The grid is partitioned into equal-sized chunks arranged in a checkerboard pattern, alternating between *black* and *white* cells similarly to squares on a chessboard.

This approach addresses multiple WFC parallelization problems, which are described in the paper:

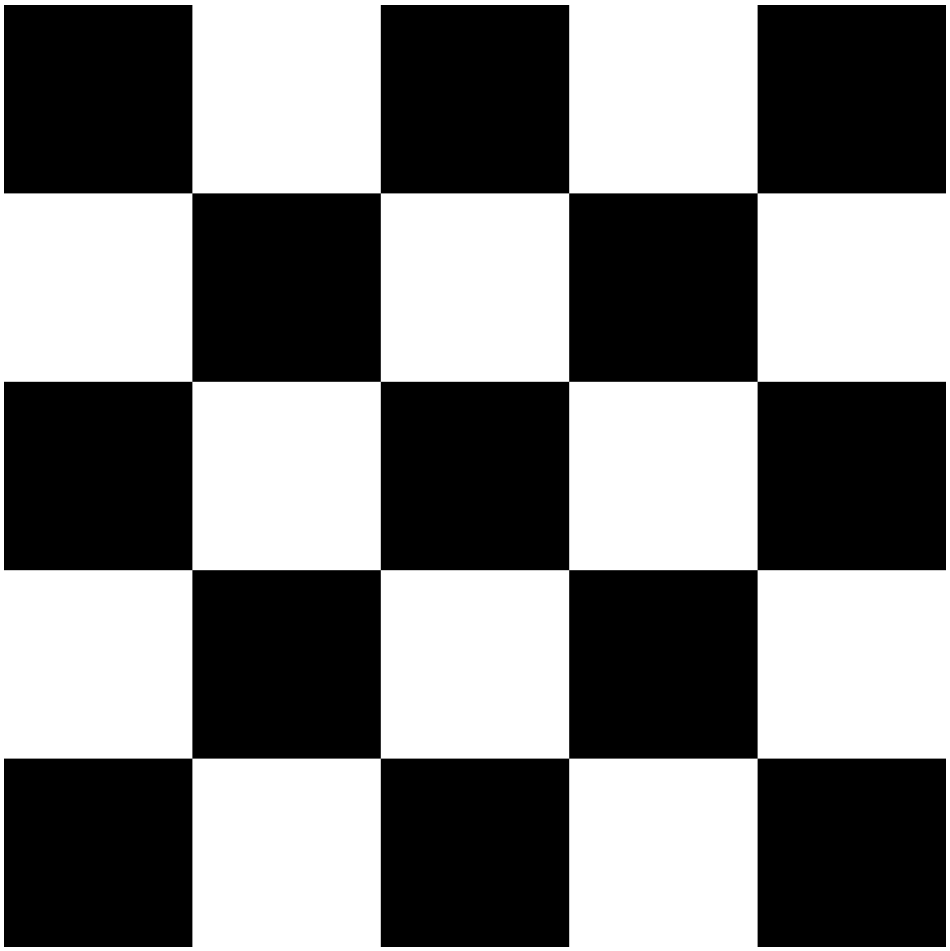

Figure 4.1: Checkboard Pattern

- Each thread works independently from any other, as each chunk is separated by other regions of another color.
- Boundary consistency is naturally achieved, as on the second pass white cells already have all 2-4 neighboring black cells generated, allowing to seam the gaps between already generated chunks. Additionally it solves already mentioned chunk seaming problem by making each *"connecting"* chunk being of the same size as already generated ones, creating space for conflict resolution during seaming process.
- Propagation step has race conditions naturally lowered and controlled, as most of the propagation happens inside of the chunks. It is important to note that propagation still occurs outside of any given processed chunk, requiring handling of race conditions during cell updates; however, due to the structural nature of generation, this happens rarely and does not meaningfully impact performance.

### Two-Pass Generation

Generation proceeds in two phases.

**Phase 1** processes all black chunks in parallel. By construction, no two black chunks are adjacent - each black chunk is surrounded entirely by white chunks. Black chunks therefore share no borders with one another, and their generation is fully independent. Each thread collapses within its assigned chunk without any inter-thread communication. However, to ensure that white chunks in `Phase 2` have possibility sets already narrowed by black chunks, propagation step may extend beyond currently processed section of the grid.

**Phase 2** processes all white chunks in parallel, beginning only after Phase 1 has completed. At this point, every cell on the boundary of every white chunk belongs to a fully collapsed black chunk. White chunks therefore begin generation with all four borders fully determined, providing complete boundary constraints from the outset.

### Contradiction Handling

Although the checkerboard structure substantially reduces the probability of contradictions, it does not eliminate them entirely. Within a single chunk, a contradiction may arise if the local constraint propagation reaches a state where no valid tile exists for some cell. In Phase 1, such contradictions are self-contained and the affected chunk can simply be regenerated independently.

In Phase 2, contradictions are more complex: a white chunk's borders are fixed by its Phase 1 neighbors, so a contradiction may reflect an over-constraining boundary rather than a local generation failure. The following resolution strategy is proposed. If a contradiction occurs in a white chunk, the chunk is regenerated up to a fixed number of attempts (three in the current design). If the contradiction persists across all attempts, at least one neighboring black chunk is likely the source of the over-constraint, and one of the adjacent black chunks is selected for regeneration, followed by a retry of the white chunk. If all neighboring chunks have been tried without success, the image is discarded and generation restarts from scratch. This case is expected to be exceedingly rare: persistent contradictions after exhausting all neighbors indicate either a globally inconsistent constraint set or a pathological tileset, neither of which can be resolved by local repair.

It should be noted that this contradiction handling strategy was not implemented in the current prototype and remains a design proposal for future work.

### Cell Size Analysis

The performance of the checkerboard approach is sensitive to chunk size. A chunk that is too small relative to the tile constraint range will have its interior dominated by border cells, increasing the seam-solving burden and contradiction rate. A chunk that is too large risks underutilizing threads.

#### 4.2.0.1 Lower Bound

For a square chunk of side length $n$ with a tile constraint range of $r = 1$, the areas of the interior and border regions are:

$\mathrm{A}_{total} = n^2$

$A_{interior} = (n-2)^2$

$A_{border} = n^2 - (n-2)^2$

Requiring the interior-to-border ratio to be at least 1 ensures that boundary effects do not dominate chunk generation:

$$\frac{(n-2)^2}{n^2 - (n-2)^2} \geq 1 \tag{4.1}$$

Rearranging:

$$n^2 - 8n + 8 \geq 0 \tag{4.2}$$

The positive root of the corresponding equality is $n = 4 + 2\sqrt{2} \approx 6.83$, yielding a minimum chunk side length of $\mathbf{n} = \mathbf{7}$.

Empirical validation of this bound and its effect on contradiction rates is left for future work.

#### 4.2.0.2 Upper Bound

Having more chunks than threads - *oversubscription* - is generally beneficial, keeping threads fed as faster chunks complete and avoiding idle time from stragglers.

Experimental results did not reveal a measurable performance degradation from large chunk sizes within the grid sizes tested, suggesting that the upper bound is not a practical concern at the scales evaluated. Whether this holds for significantly larger grids remains an open question. A principled derivation of the upper bound is left for future work.

# Chapter 5

# Implementation

Implementation was done in C++ using `OpenMP` for parallelization for its ease of usage. CPU cores were chosen for parallelization as selected algorithm, including rule propagation, is more complex and does not work well with GPU implementations like `CUDA`.

The whole grid is contained withing the shared memory during parallel execution, as algorithm design does not require private memory, which improves performance.

## Grid splitting

Grid splitting into chunks is performed to allocate an appropriate amount of regions to work with in future generation, based on already described math.

The amount of regions per Pass is calculated as follows:

$$T = C * O$$

Where:

- T - chunk amount per pass
- C - core count
- O - oversubscription ratio

Core amount is fetched using *omp_get_num_procs* function from *OpenMP*. Chunks are calculated based on core count as it allows to allocate maximum performance, while not overusing them by introducing more threads then CPU can provide, introducing unnecessary overhead.

An oversubscription factor of 2× was selected based on experimental observation, as it yielded consistent performance improvements across the tested grid sizes. A principled derivation of the optimal oversubscription factor is left for future work.

**Algorithm 1** Split Grid Into Chunks

```
function SplitGridIntoChunks(grid)
    totalChunks ← cores × oversubscription × 2
    chunksPerAxis ← ⌈√totalChunks⌉
    paddedWidth ← smallest multiple of chunksPerAxis ≥ grid.width
    paddedHeight ← smallest multiple of chunksPerAxis ≥ grid.height
    chunkWidth ← paddedWidth / chunksPerAxis
    chunkHeight ← paddedHeight / chunksPerAxis
    for y ← 0 to chunksPerAxis −1 do
        for x ← 0 to chunksPerAxis −1 do
            chunk.startX ← x× chunkWidth
            chunk.startY ← y× chunkHeight
            chunk.endX ← min(chunk.startX + chunkWidth, grid.width)
            chunk.endY ← min(chunk.startY + chunkHeight, grid.height)
            chunk.phase ← (x + y) mod 2
            add chunk to chunks
        end for
    end for
    return chunks
end function
```

Resulting chunks are then being split into two groups based on their phase, and processed in multiple steps, as described in previous chapter.

To properly allocate threads for each chunk, they are processed in a for loop, and a following OMP pragma is used: *#pragma omp parallel for schedule(dynamic)*

As memory of the grid is shared, no specifications about memory management are provided in the pragma. Dynamic scheduling is applied, as it allows to allocate already finished threads to other chunks, allocated using oversubscription during grid separation, speeding up the generation.

**Observation** step is implemented by finding and collapsing a cell with the lowest entropy, within the processed chunk. To ensure more variable results in generation each cell is chosen randomly from the set of lowest entropy grid members.

**Propagation** step is performed in a loop until there is no uncollapsed cells left in the grid. Each such loop is started within a parallelized for loop above chunks.

Rule propagation, as already stated in Chapter3, is performed using BFS. Last collapsed cell is put first into the queue for update. During a cell processing, each neighboring cell has its rules updated, based on state (available tileset and entropy) of the current cell. If any changes in available tiles of the neighboring cell occurred, it is put into queue for future processing. To avoid redundant propagation to already visited cells, they are being added to a hash map upon processing, and if those cells are already in the hash map, they are skipped and removed from the processing.

As already stated in Solution Chapter4, propagation, compared to observation, may happen outside of currently processed chunks, so it is necessary to ensure race conditions wont occur, when

**Algorithm 2** Constraint Propagation

```
function PropagateConstraints(startCell)
    queue ← empty queue
    visited ← empty set
    enqueue(queue, startCell)
    while queue is not empty do
        current ← dequeue(queue)
        if current is in visited then
            continue
        end if
        add current to visited
        for each neighbor of current do
            if neighbor does not exist then
                continue
            end if
            if neighbor entropy ≤ 1 then
                continue
            end if
            changed ← reduce neighbor's possible states using constraints from current
            if changed then
                enqueue(queue, neighbor)
            end if
        end for
    end while
end function
```

two threads will try to propagate new rules to the same cell. To implement this, function which updates available tiles in the cell, must be locked.

For locking functionality OMP's *omp_lock_t* were used.

**Algorithm 3** Update Cell Domain

```
function UpdateCell(cell, neighborTiles, direction)
    acquire lock on cell
    for each tile in neighborTiles do
        allowed ← tile.rules[direction]
        add all elements of allowed to validIDs
    end for
    newDomain ← empty set
    for each tile in cell.domain do
        if tile.id is in validIDs then
            add tile to newDomain
        end if
    end for
    if size of newDomain = size of cell.domain then
        release lock on cell
        return false
    end if
    cell.domain ← newDomain
    cell.entropy ← size of newDomain
    release lock on cell
    return true
end function
```

Additionally, the cell collapse method and the method returning the current possibility set are guarded by a lock as well, ensuring that no two threads simultaneously modify or read an inconsistent cell state during cross-boundary propagation.

```
void Cell::collapse()
{
    omp_set_lock(&lock);
    if(tiles.empty())
    {
        omp_unset_lock(&lock);
        return;
    }

    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_int_distribution<> distr(0, tiles.size()-1);
```

```
    enthropy = 1;
    shared_ptr<Tile> new_tile = tiles.at(distr(gen));
    tiles = vector<shared_ptr<Tile>> {new_tile};

    omp_unset_lock(&lock);
}
```

Passes 1 and 2 are functionally the same, rule propagation beyond currently processed chunk already provides necessary information to collapse white chunks on the second pass.

**Algorithm 4** Chunk-Based Grid Collapse

```
function ChunkGridCollapse
    initialize grid
    chunks ← split grid into chunks
    blackChunks ← empty list
    whiteChunks ← empty list
    for each chunk in chunks do
        if chunk.phase = 0 then
            add chunk to blackChunks
        else
            add chunk to whiteChunks
        end if
    end for
                                        ▷ Phase 1: process independent chunks
    parallel for each dynamic scheduling chunk in blackChunks
    changedCell ← pick random cell in chunk
    while changedCell exists do
        propagate constraints from changedCell within chunk
        changedCell ← select cell with lowest entropy in chunk
    end while
                                        ▷ Phase 2: process remaining chunks
    parallel for each dynamic scheduling chunk in whiteChunks
    changedCell ← select cell with lowest entropy in chunk
    while changedCell exists do
        propagate constraints from changedCell within chunk
        changedCell ← select cell with lowest entropy in chunk
    end while
end function
```

# Chapter 6

# Experiments

Multiple experiments were conducted to evaluate key aspects of the parallelized solution, including execution time and contradiction rate. Results were compared against the sequential implementation under otherwise identical conditions.

## 6.1 Experimental Setup

### Hardware and Software

All experiments were conducted on the following system:

| Component | Specification |
|---|---|
| CPU | AMD Ryzen 7 7840HS, 16 logical cores |
| RAM | 32 GB DDR5 |
| OS | Arch Linux |

Table 6.1: Experimental hardware configuration.

### Tileset

The *circuit* tileset was selected for all experiments. This tileset provides stable, well-defined adjacency rules with a low contradiction rate under sequential generation, while offering a large number of valid tile combinations. The tileset consists of 13 unique base images; non-symmetric tiles are automatically rotated by the implementation prior to generation to increase output variability, yielding a final working set of **47 distinct tiles**.

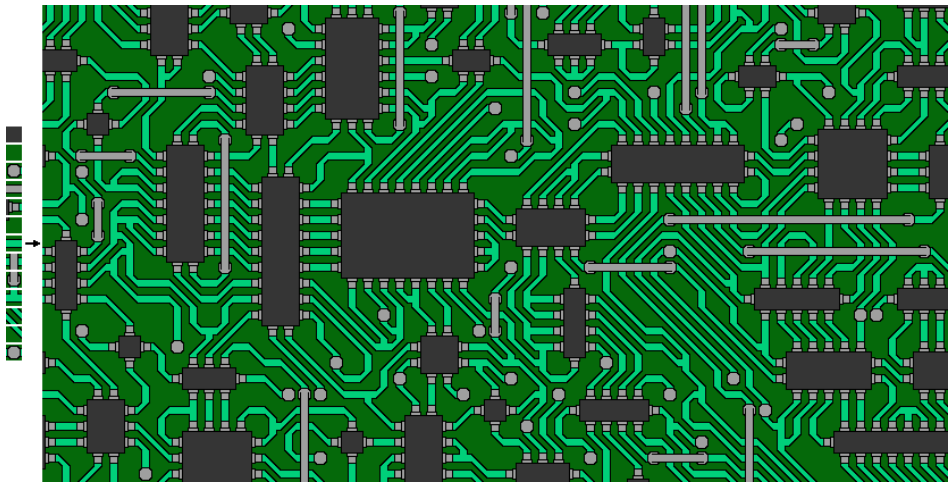

Figure 6.1: Circuit Tileset provided by Maxim Gumin [2]

### 6.1.1 Grid Sizes and Repetitions

Experiments were conducted across four grid sizes: $10 \times 10$, $20 \times 20$, $50 \times 50$, and $100 \times 100$. This range was chosen to represent realistic use cases while providing a progression of computational load; at the largest size, up to 10,000 cells must be collapsed in a single generation pass. Each configuration was evaluated over 20 independent generation attempts for both the sequential and parallel implementations.

Each grid size was evaluated under two oversubscription configurations: a factor of $1\times$ (one chunk per logical core per phase) and $2\times$ (two chunks per logical core per phase). Unless stated otherwise, figures presented in this chapter correspond to the $1\times$ configuration.

## 6.2 Metrics

The following metrics were recorded for each experiment:

- **Execution time** - time from grid initialization to final image output, in milliseconds.
- **Mean and standard deviation** of execution time across the 20 repetitions, to characterize both average performance and run-to-run variability.

## 6.3 Results

Results of testing generations with both sequential and parallel algorithms were recorded and processed.

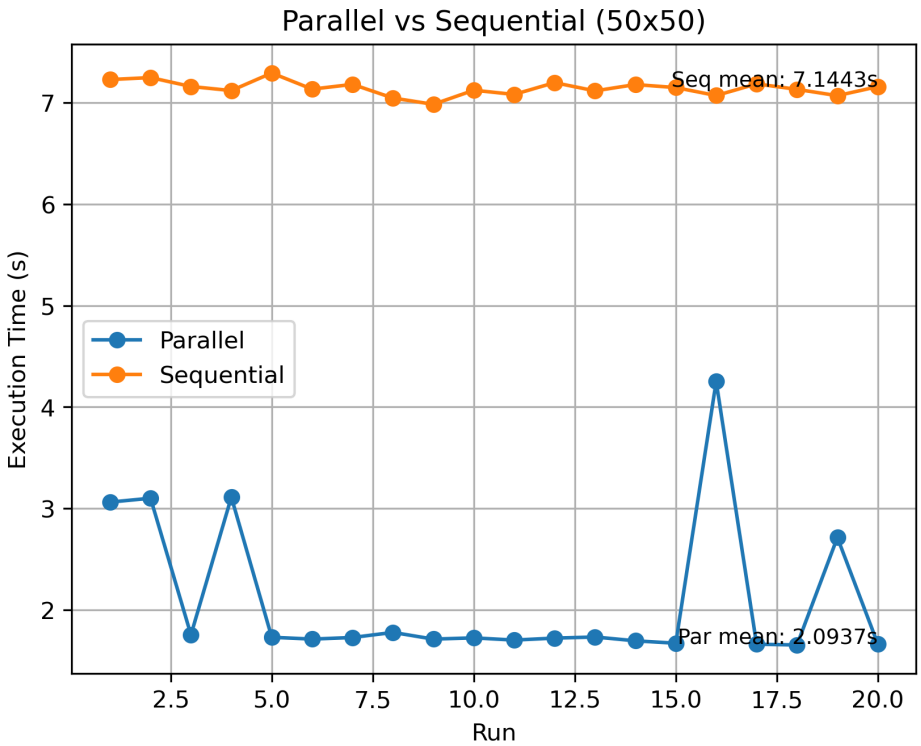


Figure 6.2: Sequential and parallel generation time through testing passes

### Speedup

Parallelization yielded a substantial improvement in execution time. Speedup increased with grid size, following an asymptotic curve consistent with Amdahl's Law [4], with a maximum observed speedup of 4× relative to the sequential baseline. Additional tests at $150 \times 150$ show 4× speedup being maximum for chosen method.

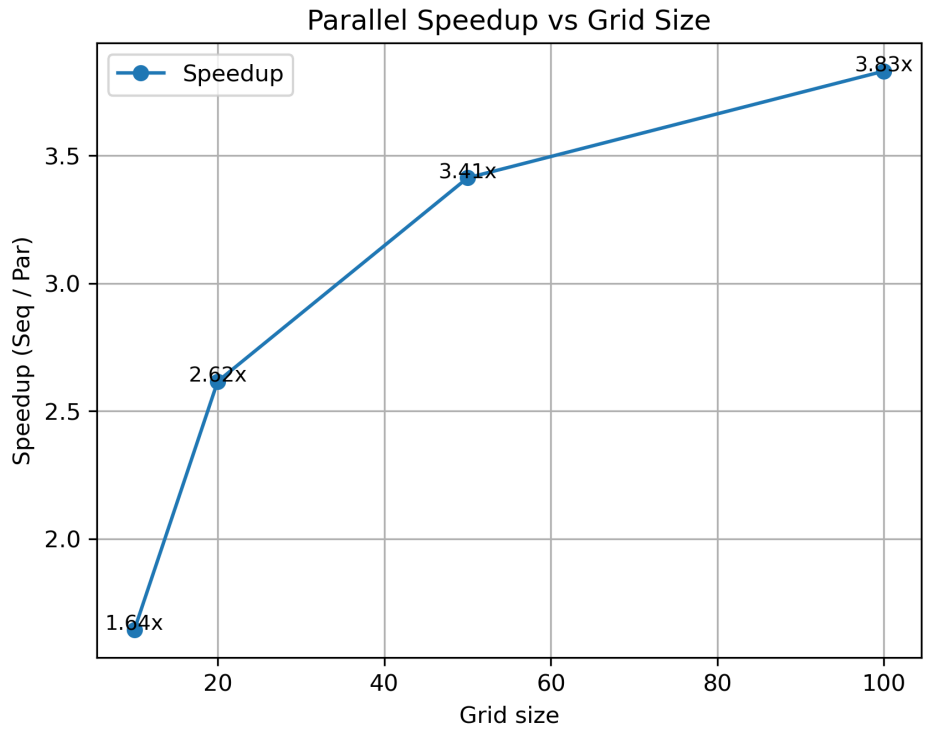


Figure 6.3: Speedup of parallel solution compared to sequential one, relative to grid size

### Parallel Efficiency

The maximum parallel efficiency observed across all experiments was:

$$E = \frac{S}{n} = \frac{3.83}{16} \approx 0.24$$

This indicates that, at peak, approximately 24% of available thread capacity was productively utilized. The relatively low efficiency is consistent with the sequential fractions of the algorithm — initialization, the observation, and contradiction handling.

### Oversubscription

An oversubscription factor of 2× yielded a performance improvement of approximately 10% across all grid sizes compared to a 1× assignment (one chunk per core). This confirms that dynamic scheduling benefits from having spare work items available when threads finish early, mitigating the impact of stragglers.

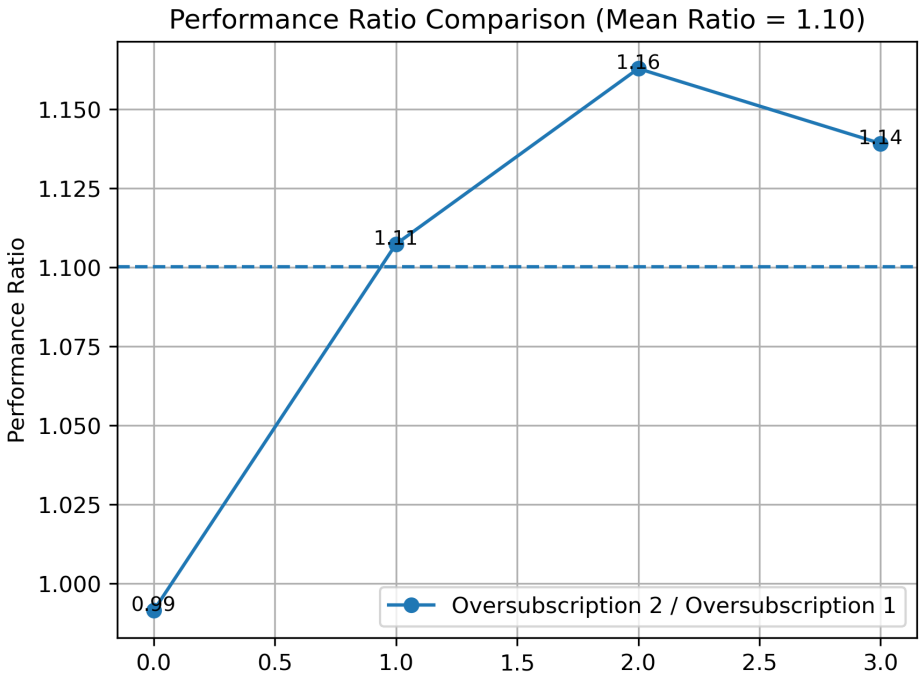


Figure 6.4: Oversubscription influence on performance

However, during testing at a grid size of $150 \times 150$, a slight regression was observed: speedup decreased from $4\times$ to $3.69\times$. Crucially, this regression was present under both oversubscription factors of $1\times$ and $2\times$, ruling out oversubscription as a contributing factor. The cause of this regression is not well understood and is left as an open question for future work.

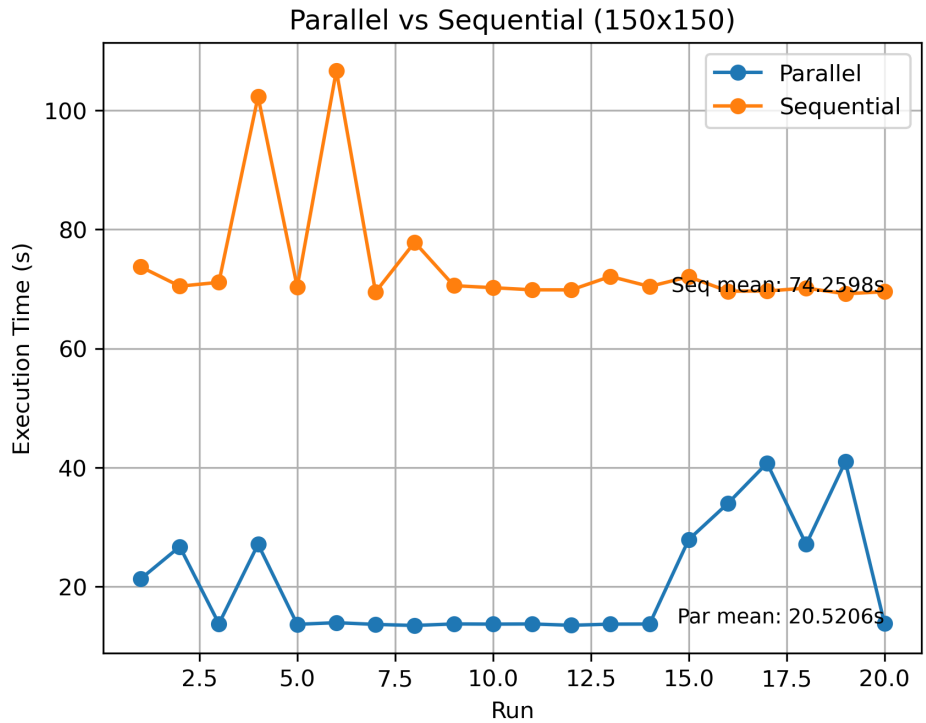


Figure 6.5: Performance of parallel and sequential algorithms on a 150x150 grid

### Parallel Fraction Analysis

Amdahl's Law [4] can be used to quantify what fraction of the algorithm executes in parallel, and to explain the observed speedup characteristics. The speedup $S$ is defined as:

$$S = \frac{T_{seq}}{T_{par}}$$

where $T_{seq}$ is the sequential execution time and $T_{par}$ is the parallel execution time. The parallel fraction $p$ can then be derived as [**grama2003**]:

$$p = \frac{1 - \frac{1}{S}}{1 - \frac{1}{n}}$$

where $n$ is the number of threads.

Applying this formula across grid sizes reveals that the parallel fraction increases with grid size, which is consistent with the expected behavior: larger grids produce more chunks, reducing the relative contribution of the fixed sequential overhead. It may further be hypothesized that this fraction also depends on tileset complexity, as richer tilesets produce longer propagation waves, increasing the proportion of time spent in the parallelized propagation step.

### Known issues

During testing, a small number of generation attempts on $10 \times 10$ grids produced visual artifacts in the output image, manifesting as incorrect tile placements at chunk boundaries that violate local adjacency rules. This is distinct from a full contradiction - rather than failing to find a valid tile, the implementation occasionally places a tile silently, without detecting the constraint violation.

The root cause was not identified within the scope of this work. A likely candidate is an undetected race condition during cross-boundary rule propagation in Phase 1, where multiple threads may concurrently modify the possibility sets of shared border cells. Notably, the issue was not observed in grids where chunk size met or exceeded the lower bound derived in Chapter 4, suggesting that the theoretical bound has practical significance beyond the error rate argument from which it was originally derived. This is left as a known defect for future investigation.

# Chapter 7

# Conclusion

In this work a checkerboard chunk-based approach to parallelizing Wave Function Collapse was described, implemented and tested. The grid is partitioned into equal-sized chunks arranged in two-color checkerboard pattern, allowing two-pass generation strategy in which black chunks are collapsed independently, and white chunks are collapsed during the second pass with fully determined boundaries and rules propagated from the first. The approach was implemented in C++ using OpenMP and evaluated against a sequential baseline across a range of grid sizes.

Experimental results demonstrated a maximum speedup of $4\times$ on a 16-core processor, with speedup increasing asymptotically with grid size in a manner consistent with Amdahl's Law. Analysis of the parallel fraction confirmed that larger grids yield a higher proportion of parallel work. An oversubscription factor of $2\times$ provided an additional 10% performance improvement at moderate grid sizes, though a slight regression was observed at $150 \times 150$ under both oversubscription configurations, ruling out oversubscription as the cause.

The current implementation has several limitations. Contradiction handling in Phase 2 was not fully implemented, leaving chunk regeneration as a design proposal rather than a tested component. Implementation instability at small grid sizes was observed but not resolved within the scope of this work. The evaluation was conducted on a single tileset and hardware configuration, and generalization to other tilesets remains to be verified.

Future work could extend the approach in several directions. Full implementation of chunk regeneration for contradiction handling would complete the prototype and improve reliability across multiple passes. Resolving the instability at small grid sizes would broaden the range of practical applicability. Additionally more thorough mathematical analysis of lower and higher chunk size bounds could be conducted.

# Literatura


1. MERRELL, Paul. Example-Based Model Synthesis. In: *Proceedings of the 2007 Symposium on Interactive 3D Graphics and Games*. New York, NY, USA: ACM, 2007, pp. 105–112. I3D '07.
2. GUMIN, Maxim. *Wave Function Collapse Algorithm* [`https://github.com/mxgmn/WaveFunctionCollapse`]. 2016.
3. ORLOWSKI, Jan; LEE, Amy. *Parallel Wave Function Collapse* [`https://github.com/amylh/WaveCollapseGen`]. 2018.
4. AMDAHL, Gene M. Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities. In: *Proceedings of the AFIPS Spring Joint Computer Conference*. AFIPS Press, 1967, pp. 483–485.